CS772 Project Report

Aastha Punjabi (210017)
Indian Institute of Technology Kanpur

Priya (210771)
Indian Institute of Technology Kanpur

Rohan Batra (210868)
Indian Institute of Technology Kanpur

Sharvani (210960)
Indian Institute of Technology Kanpur

Abstract

This project aims to improve training efficiency, calibrate the model confidence, and improve the predictive uncertainty on the outlier data. We first filter data using a prioritised data selection strategy, called RHO-LOSS, which focuses on points that are learnable, not yet learnt, and worth learning. This helps to increase the training efficiency and in practice is observed to obtain a greater training accuracy faster. The DNN obtained thereafter is then converted to a BNN using the ABNN framework. This allows us to equip the normal DNN with uncertainty predictions and in practice is obtained to improve the performance on the NLL metric.

1 Problem description/motivation

The primary objective of this project is to systematically address the critical challenges encountered when training deep neural networks (DNNs) for tasks involving extremely large datasets.

When working with vast amounts of internet-scraped data, the key issues faced during training are that it often consumes an excessive amount of time, with most computational resources being wasted on redundant and noisy points that have already been learned or are not learnable. To address this issue, we use training on a selection of data points in batches based on a novel loss function (RHO-LOSS) instead of uniform selection. The loss function is designed to identify and reject redundant, noisy, and less relevant points.

Another challenge faced by DNNs is the absence of predictive uncertainties, which limits their practical applicability in real-world scenarios. While BNNs offer a promising alternative, they can be unstable to train on large-scale DNNs. To address this issue, we aim to adapt a large DNN into a BNN by transforming its normalization layers into Bayesian normal layers and then fine-tuning the network for a few iterations. This process enables us to obtain uncertainty quantification.

The overall objective of this project was to develop a comprehensive pipeline that can effectively address both these challenges and provide us with an efficiently trained uncertainty estimate equipped with a deep neural network.

2 Literature review and description of prior work on the problem

2.1 Reducible Holdout Loss Selection (RHO-LOSS)

Reducible Holdout Loss Selection (RHO-LOSS) is a data selection method designed to accelerate neural network training by prioritizing datapoints that are learnable, worth learning, and not yet learnt.

Prior work, such as loss-based selection (Loshchilov Hutter, 2015) and gradient norm-based methods (Katharopoulos Fleuret, 2018), prioritizes "hard" points (e.g., high loss) to avoid redundant training has been done. However, these methods often select noisy or irrelevant points, degrading performance. Web-scraped datasets (e.g., Clothing-1M) often contain mislabeled or ambiguous samples. Prior work (Chen et al., 2019; Pleiss et al., 2020) focuses on identifying noisy labels but does not integrate this into online training.

RHO-LOSS avoids noisy points by deprioritizing high irreducible loss (IL) points, likely unlearnable due to noise. It skips Redundant points by focusing on points with high reducible loss (training loss minus IL).

RHO-LOSS is derived from a probabilistic framework but uses approximations (e.g., SGD instead of exact Bayesian inference) for tractability. It directly targets holdout loss, aligning with generalization goals. Combines insights from Bayesian modeling and online selection to derive a principled objective. Outperforms baselines on noisy (Clothing-1M) and clean datasets (CIFAR, NLP tasks), with 18x speedup and +2$\%$ accuracy. RHO-LOSS advances data selection by unifying three criteria: learnability, relevance, and non-redundancy. Its empirical success on diverse tasks (vision, NLP) and robustness to noise position it as a scalable solution for modern large-scale training.

$$\text{RHO - LOSS}(x, y) = \underbrace{L[y \mid x; \mathcal{D}_t]}_{\text{Training Loss}} - \underbrace{L[y \mid x; \mathcal{D}_{\text{ho}}]}_{\text{Irreducible Holdout Loss}} \tag{1}$$

Where:

- $L[y|x; D_t]$ is the training loss on the current model (trained on data D_t).
- $L[y|x; D_{h_o}$ is the irreducible holdout loss, computed using a model trained on the holdout set D_{h_o}.

2.2 Adaptable Bayesian Neural Networks (ABNN)

Make Me a BNN paper proposes a novel, scalable approach for Bayesian uncertainty estimation that avoids the high training and inference costs of traditional BNNs and ensemble methods.

We use this technique to transform a pre-trained DNN into a Bayesian Neural Network (BNN) in a post-hoc manner by attaching a simple plug-in module to the normalization layers and applying only a few epochs of fine-tuning. The resulting model, called **Adaptable Bayesian Neural Network (ABNN)**, achieves excellent performance on various vision tasks without incurring the high computational cost associated with ensemble methods. ABNN is compatible with multiple DNN architectures, including ResNet and Vision Transformers.

Classical Bayesian Neural Networks (BNNs) use a posterior distribution over network weights to model epistemic uncertainty. The predictive distribution is obtained via marginalization:

$$P(y \mid x, D) = \int P(y \mid x, \omega) P(\omega \mid D)\, d\omega \tag{2}$$

This integral is intractable in practice.In our approach, we use **variational inference**, which approximates the posterior $P(\omega \mid D)$ with a simpler distribution. This leads to scalable methods like variational BNNs using the reparameterization trick:

$$\omega = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \tag{3}$$

The ABNN method transforms a pre-trained deterministic DNN into a Bayesian model by modifying its normalization layers to act as **Bayesian Normalization Layers (BNLs)**. Each normalization layer (e.g., BatchNorm, LayerNorm, or InstanceNorm) is modified and some Gaussian noise is injected during training and inference. The modified activation layer becomes:

$$\text{BNL}(h_j) = \frac{h_j - \mu_j}{\sqrt{\sigma_j^2 + \delta}} \cdot (1 + \epsilon_j) + \omega_j \tag{4}$$

In this method, only a few trainable parameters are added (associated with noise vectors and layer-wise variances) that are then fine-tuned for a few epochs. This constructs the ABNN 3–5 times **faster** than training full ensembles or variational BNNs.

The model introduces a class-weighted perturbation term in the loss to **avoid mode collapse** when training multiple ABNN instances. Thus, it achieves ensemble diversity and **multi-modality** by retraining only stochastic normalization parameters, which reduces gradient variance and stabilizes training compared to classical BNNs.

During inference, uncertainty is captured by averaging predictions over multiple noise instantiations (ε-samples) and model instances (ω-samples), closely approximating a Bayesian posterior ensemble.

2.3 Out-of-distribution (OOD) detection

Out-of-distribution (OOD) detection is important for the safe deployment of machine learning models. Traditional methods often use auxiliary outlier datasets, but collecting such data is labor intensive. Recent approaches like VOS and NPOS generate synthetic outliers in feature space but lack human-interpretable results.

DREAM-OOD introduces a novel method to generate photo-realistic outliers using diffusion models, that are conditioned only on in-distribution (ID) data. By learning a text-conditioned latent space and sampling from low-likelihood regions, DREAM-OOD creates meaningful outliers that improve OOD detection.

To enhance training, DREAM-OOD proposes an additional loss, L_{ood}, to separate in-distribution (ID) and out-of-distribution (OOD) data during classification. It is defined as:

$$\begin{aligned} L_{\text{ood}} = \mathbb{E}_{x_{\text{ood}}} &\left[-\log \left(\frac{1}{1 + \exp\left(\phi\left(E\left(f_\theta(x_{\text{ood}})\right)\right)\right)} \right] \right. \\ &+ \mathbb{E}_{x \sim P_{\text{in}}} \left[-\log \left(\frac{\exp\left(\phi\left(E\left(f_\theta(x)\right)\right)\right)}{1 + \exp\left(\phi\left(E\left(f_\theta(x)\right)\right)\right)} \right] \right. \end{aligned} \tag{5}$$

where $f_\theta(x)$ is the classifier output, $E(\cdot)$ is the energy function, and $\phi(\cdot)$ is a small multilayer perceptron. This loss encourages low confidence on synthetic OOD images and high confidence on ID images, improving detection performance.

Compared to previous methods, DREAM-OOD produces high-resolution, interpretable images and has better performance on CIFAR-100 and ImageNet-100 datasets.

3 Novelty of our work

Our pipeline integrates three techniques - RHO-LOSS, ABNN, and DREAM-OOD into a cohesive framework for efficient and reliable deep learning. We implemented a ResNet18 architecture from scratch and replaced its standard loss function with the RHO-LOSS function to prioritize training on data points that are learnable, not yet learned, and worth learning. After training this filtered model, we converted all normalization layers into Bayesian normalization layers to enable uncertainty estimation. We retrained the modified model on the complete dataset with fewer epochs, leveraging the improved initialization and uncertainty-aware structure. This modification enhances the model's adaptability to diverse inputs.

4 Pipeline of the project

Algorithm 1 Training Pipeline (RHO-LOSS + ABNN)

Input: Dataset $\mathcal{D}$, holdout set $\mathcal{D}_{ho}$, learning rate η, batch sizes n_b, n_B, epochs E_1, E_2
Train small model $p(y \mid x; \mathcal{D}_{ho})$ on holdout set
for $(x_i, y_i) \in \mathcal{D}$ **do**
    $\texttt{IrreducibleLoss}[i] \leftarrow -\log p(y_i \mid x_i; \mathcal{D}_{ho})$
end for
Initialize model θ^0, $t \leftarrow 0$
for $e = 1$ to E_1 **do**
    Sample large batch B_t of size n_B
    Compute $Loss[i]$ for each $x_i \in B_t$ using θ^t
    Compute $RHOLOSS[i] \leftarrow Loss[i] - \texttt{IrreducibleLoss}[i]$
    Select $b_t \leftarrow$ top-n_b samples with highest RHOLOSS
    Compute gradient g_t on b_t and update: $\theta^{t+1} \leftarrow \theta^t - \eta g_t$
    $t \leftarrow t + 1$
end for
Replace all normalization layers in θ with Bayesian Normalization Layers (BNL)
for $e = 1$ to E_2 **do**
    for $(x, y) \in \mathcal{D}$ **do**
        Forward pass with stochastic BNL
        Compute uncertainty-aware loss $L(\theta, x, y)$
        Update parameters using gradient descent
    end for
end for

5 Description of tools/software used

- Google Colab, Jupyter Notebook and VS Code: Environments for running and working with python scripts and notebooks
- Conda for environment and packages management
- Hydra for config file management
- Weights and Biases (Wandb) for logging and saving run data
- IITK CSE GPU Server

6 Experimental results and data description

6.1 Data Used

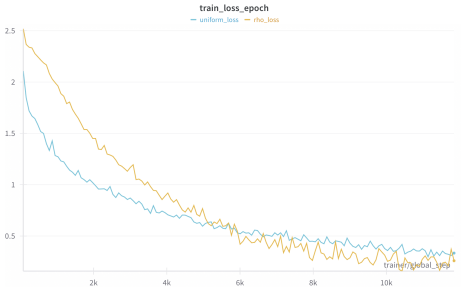
We used CIFAR-10 and CIFAR-100 datasets with a holdout split for irreducible loss estimation.

6.2 Results

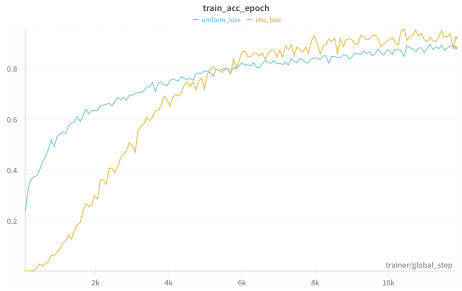
6.2.1 RHO-LOSS sped up training and improved final accuracy

Here, uniform selection refers to randomly (uniformly) choosing a set of points from the current batch and training the model (updating weights) based on them. Whereas RHO-LOSS refers to choosing points with the highest RHO-LOSS value from the current batch and then training on them. We observe that the RHO-LOSS trained model achieves higher accuracy and lower loss (on both training and validation), and it does so at a much faster rate (fewer iterations) for both the CIFAR-10 and CIFAR-100 Datasets. However, the difference is more pronounced on the larger CIFAR-100 Dataset.

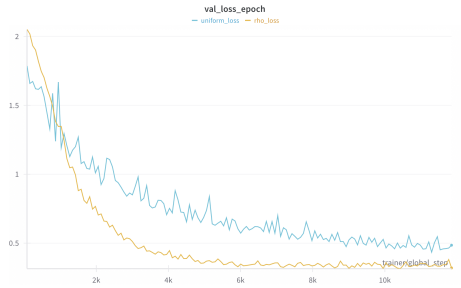
Results on CIFAR 10:



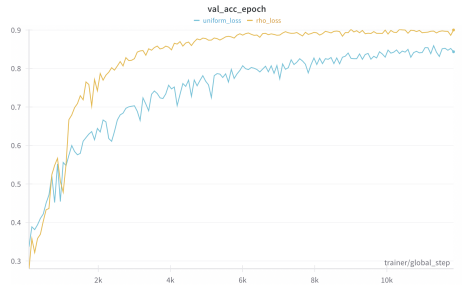
(a) Training Loss



(b) Training Accuracy



(c) Validation Loss



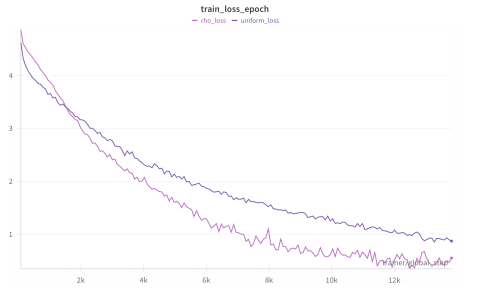
(d) Validation Accuracy

Figure 1: Comparison of uniform selection (blue curves) and RHO-LOSS based selection (yellow curves) methods on training Res-Net 18 on CIFAR10

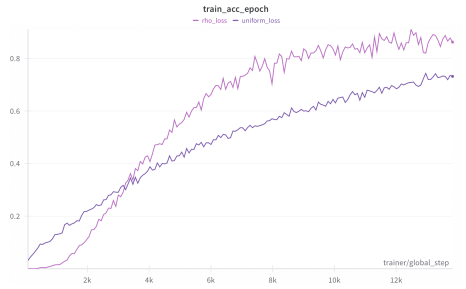
	Uniform Selection	RHO Loss Selection
Accuracy on Test Set	85.67 %	89.23%

Table 1: Test Accuracy comparison on CIFAR 10

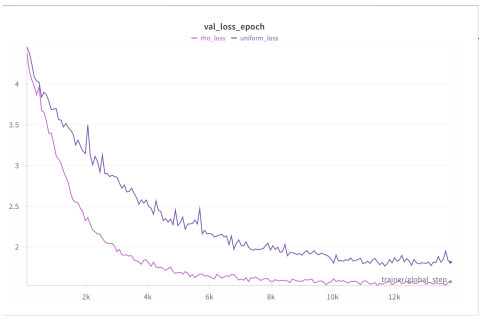
Results on CIFAR 100:



(a) Training Loss



(b) Training Accuracy



(c) Validation Loss

(d) Validation Accuracy

Figure 2: Comparison of uniform selection (purple curves) and RHO-LOSS based selection (pink curves) methods on training Res-Net 18 on CIFAR100

	Uniform Selection	RHO Loss Selection
Accuracy on Test Set	56 %	57.78%

Table 2: Test Accuracy comparison on CIFAR 100

6.2.2 ABNN added calibrated predictive uncertainty with minimal overhead

Here we compare the predictive uncertainties of the DNN (Res-Net18 model trained using RHO LOSS) with the A-BNN Model (the DNN model 'adapted' into the BNN) ensemble predictions using the Negative-log-likelihood (NLL) (the lower the better) and accuracy metrics.We observe that minimal retraining of the DNN (20 epochs) with the Bayesian Normalisation Layers noticeably improves the predictive uncertainty estimates of the model.

	DNN Model	A-BNN Model
Accuracy on Test Set	89.23 %	87.08%
NLL Test Set	1.78	0.59

Table 3: Test Accuracy comparison on CIFAR 10

	DNN Model	A-BNN Model
Accuracy on Test Set	57.78 %	57.76%
NLL Test Set	4.07	1.93

Table 4: Test Accuracy comparison on CIFAR 100

7 Things we learned

- Reading, reviewing, and analysing research papers
- Working with remote server GPUs to train ML models
- Learning to work (package , dataset installation, etc.)with open source paper implementations (official and unofficial) for verifying the results
- Training DNNs from scratch and working with checkpoints
- Efficient data selection can significantly speed up training
- The RHO LOSS paper taught how to construct an objective function mathematically by incorporating one's intuition, simplifying it into a practically usable framework through some simplifications, and then showing that they have very minimal tradeoffs
- Converting a DNN to a BNN to obtain predictive uncertainties
- ABNN algorithm teaches the 'engineering' tradeoff , how often going for fledged theoretical models isn't often required, and incorporating essential changes can provide reasonable results

8 Possible future work

- Using the DREAM-OOD paper to generate outliers and then regularising the model with the outliers. This will calibrate the model predictions and reduce the over-confidence on outliers (a problem often faced with DNNs).
- Extending and testing on larger datasets like ImageNet.
- Automate hyperparameter optimization.
- Use lightweight irreducible loss models for faster deployment.

9 References

[1] Mindermann, S. et al. (2022). "Prioritized Training on Points that are Learnable, Worth Learning, and Not Yet Learnt." *Proceedings of the International Conference on Machine Learning (ICML)*, 139, 1–12. [Online]. Available: https://proceedings.mlr.press/v162/mindermann22a/mindermann22a.pdf

[2] Franchi, G. et al. (2024). "Make Me a BNN: A Simple Strategy for Estimating Bayesian Uncertainty from Pre-trained Models." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 1–12. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2024/papers/Franchi_Make_Me_a_BNN_A_Simple_Strategy_for_Estimating_Bayesian_CVPR_2024_paper.pdf

[3] Du, X., Sun, Y., Zhu, X., Li, Y. (2023). "Dream the Impossible: Outlier Imagination with Diffusion Models." Proceedings of the Neural Information Processing Systems (NeurIPS), 1–19. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/bf5311df07f3efce97471921e6d2f159-Paper-Conference.pdf

[4] Abtin M. (2024). MakeMe-BNN [Software]. GitHub. https://github.com/abtinmU/MakeMe-BNN

[5] OATML. (2023). RHO-Loss: Robust Heteroscedastic Loss for Uncertainty Estimation [Software]. GitHub. https://github.com/OATML/RHO-Loss

10 Work Distribution

- Reading and understanding RHO LOSS paper [1] - Rohan Batra
- Training Res-Net 18 on CIFAR 10 and CIFAR 100 using RHO LOSS- Rohan Batra and Sharvani
- Reading and understanding the ABNN paper [2] - Priya and Sharvani
- Converting the trained DNNs into ABNN - Aastha Punjabi, Priya
- Reading and understanding DREAM OOD paper [3]- Aastha Punjabi
- Implementing the DREAM OOD Paper - Sharvani and Priya
- Combining the RHO-loss and ABNN paper - Rohan and Aastha

Note: We also attempted to implement the DREAM-OOD paper and were successful in most parts, including training the image embeddings. However, we were ultimately unable to fully integrate it into the code.

11 Appendix

Please find below (hyperlinks) the code notebooks for our implementation. Environment requirements are same as mentioned in the respective papers.

- RHO LOSS on CIFAR 10
- RHO LOSS on CIFAR 100
- CIFAR 10 DNN to ABNN
- CIFAR 100 DNN to ABNN

Disclaimer

We declare that this work is original and has not been submitted elsewhere for academic credit or publication. All sources are properly cited.